# TIR Liability Coding — Plain-Language Overview

For the working session. No background in machine learning assumed.

## What it does

A coder reads a TIR description and decides which QPS codes it belongs under. This program reads the same description and suggests those codes, each with a confidence. Anything below a set bar goes to a person rather than being guessed at quietly. It learned from **90,960 TIRs** — three years of the team's own decisions. Not every record carries every code, so each field learns from the ones that do.

| FIELD | CODES | LEARNED FROM | WHAT IT IS |
|---|---|---|---|
| Metric Cat | 7 | 90,958 | The kind of quality issue |
| Process Cat | 27 | 61,263 | The discipline it belongs to |
| Process Sub | 144 | 60,692 | The sub-discipline |
| Process Level 3 | 543 | 55,069 | The specific finding |

The last two are chosen **inside** the one above. Decide a TIR is `HA Hanger` and it picks only from sub-codes belonging under hangers — a combination QPS would reject cannot be produced. Those counts are what it can learn; the full QPS lists are 179 and 1,080, but the rest are used too rarely to learn from.

## The four methods

It does not have one way of deciding. It has four, they compete, and the ones that do best are kept.

**Support vector machine.** Every description is a dot on a map, placed by the words it contains; this draws the dividing lines between categories as far as possible from the nearest examples either side, so a borderline TIR still falls on the right side. **Usually wins here.**

**Stochastic gradient.** The same kind of line, reached differently — it looks at a handful of TIRs at a time and nudges the line after each. A different route settles somewhere slightly different, which is why both are kept. **It beat the support vector machine on Process Cat.**

**Logistic regression.** No line. It scores every word for and against every category — "hanger" towards `HA Hanger`, "cable" against — and adds up the words in a description.

**Gradient boosting.** A chain of small yes-or-no questions (*does it mention a weld?*), each chosen to fix what the previous ones got wrong. **The weakest here:** it must pick individual words to ask about, and there are 183,633.

**How they combine.** All four learn the field; each is scored on TIRs it has never seen; anything failing to beat the support vector machine is discarded. If more than one survives, the program works out **how much to trust each** — two survivors are not automatically equal.

## Reading the training output

Running `python -m src.train` prints this.

```
Step 1 of 3   Reading the coded TIRs
              65,718 to learn from, 11,598 held back to mark the work against
Step 2 of 3   Learning the vocabulary of the descriptions
              183,633 words and word-fragments found across 65,718 TIRs
```

Records are split, and it never sees the held-back ones while learning, so scores are earned rather than remembered. "Word-fragments" because it learns pieces of words too, so a typo still resembles the word it meant.

```
  [2 of 4]  Process Cat
            27 categories, learned from 44,159 coded TIRs
            Comparing methods, keeping whichever scores best:
              Support vector machine.....  71.1
              Stochastic gradient........  72.2
              Logistic regression........  70.6
              Gradient boosting..........  65.3
            Using:  support vector machine, stochastic gradient, blended
            Blend:  support vector machine 45%, stochastic gradient 55%  (+0.9)
```

The four scores are marks out of 100 on TIRs none of them saw. Two beat the rest, so both are kept — and the blend line says the split was worked out, not assumed: 45/55, worth **0.9 points more** than trusting them equally. The score counts every category equally, so one coded daily and one coded twice a year weigh the same; it is deliberately harsher than "percent correct".

```
  [3 of 4]  Process Sub
            144 categories, learned from 43,740 coded TIRs
            25 groups of codes: 20 learned from examples,
                                5 had only one possible answer
```

For the deeper fields it builds one small model per parent category rather than one large one. Some parents have a single possible sub-code, so no model is needed.

## What confidence means

Every suggestion carries a number from 0 to 100, and it is a real probability: 80 means it expects to be right about eight times in ten on TIRs it is that sure about. That lets the team set a policy rather than a hope.

| FIELD | CODES AUTOMATICALLY | AND IS RIGHT | LEFT FOR A CODER |
|---|---|---|---|
| Metric Cat | 94% | 95% | 6% |
| Process Cat | 80% | 95% | 20% |
| Process Sub | — | — | *cannot reach 95%* |
| Process Level 3 | — | — | *cannot reach 95%* |

The two deepest fields top out near 92% and 94% even when only the most confident answers are kept. They can help a coder; they should not code unwatched.

## What changed from the prototype

| | PROTOTYPE | NOW |
|---|---|---|
| Spreadsheets read | One layout | Any of the three QPS produces |
| Fields coded | 2 | 4 |
| Codes fit together? | Not checked | Guaranteed by construction |
| Confidence | Could not be trusted | A real probability, set per field |
| Single TIRs | Nothing kept | Collected, exported as one spreadsheet |
| Choosing a method | Three fixed, never compared | Four compete; only what earns its place |
| Training data | One export | Both, 12,321 duplicate records removed |

Three things surfaced along the way. The two exports **overlap almost entirely** — the smaller is 99.4% inside the larger — but name their columns differently, so nothing noticed: the same TIRs would have been learned from twice, then used to mark the program's own work. **A third of records had no Process Cat**, and the prototype was learning "blank" as a real category; it had become the largest category in the data. And **coders disagree with each other** — where the same description was coded more than once, the codes conflict:

| FIELD | CODES CONFLICT | MODEL IS AT |
|---|---|---|
| Metric Cat | 32% | 92.8% |
| Process Cat | 31% | 88.2% |
| Process Sub | 43% | 80.8% |
| Process Level 3 | 52% | 73.6% |

That last one is the most important finding, and it is not a criticism — some TIRs are genuinely ambiguous. But it **sets a ceiling**, and the program is already at it on every field. Better numbers now depend on agreeing the codes, not on better software.

## Two questions from review

**"Why only Description 1?"** For coding, it is not — the program reads Description 1, Description 2 and Doc Title together, worth 5.4 points. Only the *agreement* figures group on Description 1 alone, because matching on all three makes near-identical TIRs look distinct: the comparable set drops **eleven-fold**, from 932 repeated Process Cat descriptions to 83. Two TIRs sharing a Description 1 can be different events, so those agreement figures are a **floor** — real agreement is somewhat better.

**"Why build the hierarchy from what coders did, rather than the code numbering?"** The codes look like they nest — `HA` → `HAPI` → `HAPIFK` — but records follow that only 99.89% of the time at the second level and **95.51% at the third: 2,555 records disagree.** Building from the numbering would rule those real combinations out and admit combinations nobody has used in three years. *Those 2,555 are either a coding pattern worth understanding or a data-quality issue worth fixing — the data cannot say which.*

---

## What would improve it

1. **Settle the conflicting codes.** 291 descriptions were given two different Process Cats. That list can be handed over as a spreadsheet — a day or two of work, and the only thing that raises the ceiling.
2. **Write down the tie-breakers.** One page of *"when it is both X and Y, code X"* for the pairs coders always stop and think about helps people and program equally.
3. **Retire the 537 Level 3 codes** used fewer than ten times in three years.
4. **Confirm the Process Cat before the deeper codes are chosen.** Already in the app. Lifts the sub-code from 81% to 90% — one click, worth more than every software change tried.

**In short:** it codes Metric Cat and Process Cat about as consistently as the team does, and can take roughly 80% of Process Cat off the queue at 95% accuracy. The two deepest levels are genuinely hard — for people as much as for software — and should stay advisory. Nothing further in the software moves these numbers much; agreement about what the codes mean will.